

Práctica de JOINS — Semana 1

 SQL BASE

PRÁCTICA COMPLEMENTARIA

Bootcamp: Fundamentos de Ingeniería de Datos · Bloque temático: SQL BASE

Dominar los diferentes tipos de JOINS en SQL

Dataset: Tablas temporales creadas en el notebook · Instructor: Luciano Argolo

W

Instrucciones

01

Crear el notebook

Crea un nuevo notebook en Databricks llamado `Semana_1_Practica_JOINS`

02

Configurar celdas SQL

Usa `%sql` al inicio de cada celda SQL para activar el modo SQL en Databricks.

03

Crear las tablas de práctica

Ejecuta primero las celdas que crean las tablas: `clientes`, `productos`, `ordenes`, `empleados`.

04

Resolver ejercicios paso a paso

Avanza por cada ejercicio en orden, verificando que los resultados tengan sentido lógico.



Tips importantes: Si te trabas, revisa los ejemplos del notebook de solución.

Presta atención a las **Preguntas de reflexión** – son clave para entender JOINS. Verifica tus resultados: ¿tienen sentido las filas que aparecen y las que no?

EJECUTAR UNA VEZ



Setup: Tablas de Práctica

Ejecuta estas celdas antes de comenzar los ejercicios. Crean el schema y las 4 tablas necesarias para practicar todos los tipos de JOINS.



clientes

6 registros · ciudades de Argentina · incluye cliente sin órdenes (Pedro)



productos

8 productos · categorías: Electrónica, Audio, Accesorios, Muebles · incluye producto nunca vendido (Silla Gamer)



ordenes

11 órdenes · rango: ene-may 2024 · incluye cliente_id=99 y producto_id=999 inexistentes



empleados

8 personas · jerarquía de 3 niveles · CEO sin jefe (para SELF JOIN)

Bloque 0 – Catalog:

```
CREATE CATALOG IF NOT EXISTS bootcamp;
```

Bloque 1 – Schema:

```
CREATE SCHEMA IF NOT EXISTS bootcamp.practice;
```

Bloque 2 – Tabla clientes:

```
CREATE OR REPLACE TABLE bootcamp.practice.clientes (  
  cliente_id INT,  
  nombre  STRING,  
  email   STRING,  
  ciudad  STRING  
);  
  
INSERT INTO bootcamp.practice.clientes VALUES  
  (1, 'Ana', 'ana@email.com', 'Buenos Aires'),  
  (2, 'Carlos', 'carlos@email.com', 'Córdoba'),  
  (3, 'María', 'maria@email.com', 'Buenos Aires'),  
  (4, 'Diego', 'diego@email.com', 'Rosario'),  
  (5, 'Laura', 'laura@email.com', 'Mendoza'),  
  (6, 'Pedro', 'pedro@email.com', 'Buenos Aires'); -- Sin órdenes
```

Bloque 3 – Tabla productos:

```
CREATE OR REPLACE TABLE bootcamp.practice.productos (  
  producto_id INT,  
  nombre  STRING,  
  categoria STRING,  
  precio  DOUBLE  
);  
  
INSERT INTO bootcamp.practice.productos VALUES  
  (101, 'Laptop Pro', 'Electrónica', 1200.00),  
  (102, 'Mouse Inalámbrico', 'Electrónica', 35.00),  
  (103, 'Teclado Mecánico', 'Electrónica', 89.00),  
  (104, 'Monitor 27"', 'Electrónica', 450.00),  
  (105, 'Auriculares BT', 'Audio', 75.00),  
  (106, 'Webcam HD', 'Accesorios', 55.00),  
  (107, 'Hub USB-C', 'Accesorios', 42.00),  
  (108, 'Silla Gamer', 'Muebles', 380.00); -- Nunca vendida
```

Bloque 4 — Tabla ordenes:

```
CREATE OR REPLACE TABLE bootcamp.practice.ordenes (
  orden_id INT,
  cliente_id INT,
  producto_id INT,
  cantidad INT,
  fecha DATE
);

INSERT INTO bootcamp.practice.ordenes VALUES
(1001, 1, 101, 1, '2024-01-15'),
(1002, 1, 102, 2, '2024-01-15'),
(1003, 2, 103, 1, '2024-02-01'),
(1004, 3, 101, 1, '2024-02-10'),
(1005, 3, 105, 3, '2024-02-10'),
(1006, 4, 104, 1, '2024-03-05'),
(1007, 5, 106, 2, '2024-03-12'),
(1008, 5, 107, 1, '2024-03-12'),
(1009, 2, 102, 1, '2024-04-01'),
(1010, 99, 101, 1, '2024-04-15'), -- cliente_id=99 NO existe
(1011, 1, 999, 2, '2024-05-01'); -- producto_id=999 NO existe
```

Bloque 5 — Tabla empleados:

```
CREATE OR REPLACE TABLE bootcamp.practice.empleados (
  empleado_id INT,
  nombre STRING,
  cargo STRING,
  jefe_id INT
);

INSERT INTO bootcamp.practice.empleados VALUES
(1, 'Roberto', 'CEO', NULL),
(2, 'Silvia', 'Directora de Datos', 1),
(3, 'Martín', 'Director Comercial', 1),
(4, 'Lucía', 'Data Engineer Sr', 2),
(5, 'Pablo', 'Data Engineer Jr', 4),
(6, 'Camila', 'Data Analyst', 2),
(7, 'Tomás', 'Vendedor Sr', 3),
(8, 'Valentina', 'Vendedora Jr', 3);
```



Recuerda usar `%sql` al inicio de cada celda en Databricks para activar el modo SQL.

Fundamentos de Ingeniería de Datos

Este material es parte del bootcamp oficial. Todos los ejercicios están diseñados para construir comprensión profunda de los JOINS en SQL, uno de los conceptos más importantes en el trabajo con bases de datos relacionales.

www.LucianoArgolo.com

Crear Datos de Práctica

Tabla de Clientes

Objetivo: Crear la tabla de clientes con los campos: cliente_id, nombre, email, ciudad.

Tabla de Productos

Objetivo: Crear la tabla de productos con los campos: producto_id, nombre, categoria, precio.

Tabla de Empleados

Objetivo: Crear tabla con auto-referencia para SELF JOIN.

Tabla de Órdenes

Objetivo: Crear la tabla temporal de órdenes con relaciones entre clientes y productos.

Validación de Tablas

Validación de datos y tablas

Objetivo: Una vez hayas creado las tablas

www.LucianoArgolo.com

Datos listos — ¡A practicar JOINS!

Con las cuatro tablas creadas, ya tienes todo lo necesario para explorar INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN y SELF JOIN en los ejercicios que siguen.

www.LucianoArgolo.com

INNER JOIN

El **INNER JOIN** devuelve únicamente las filas que tienen coincidencia en **ambas** tablas. Es el tipo de JOIN más común en reportes de datos relacionados.

Ejercicio 2.1 — Clientes con sus órdenes

Objetivo: Entender INNER JOIN básico.

Pregunta: Muestra todos los clientes que tienen órdenes, junto con los detalles. Incluye: `cliente_id`, nombre del cliente, ciudad, `orden_id`, `producto_id`, cantidad, fecha.

 **Pista:** Usa **INNER JOIN** entre `clientes` y `ordenes` usando `cliente_id`

 **Reflexión:** ¿Por qué Pedro (`cliente_id=6`) no aparece en los resultados?

Ejercicio 2.2 — Órdenes con detalle de producto

Objetivo: Hacer JOIN con múltiples tablas.

Pregunta: Muestra todas las órdenes con el nombre del producto y calcula el total (cantidad × precio). Incluye: `orden_id`, fecha, nombre del producto, categoría, cantidad, precio unitario, total.

 **Pista:** Haz **INNER JOIN** entre `ordenes` y `productos` usando `producto_id`

 **Reflexión:** ¿Por qué la orden 1011 no aparece?

Ejercicio 2.3 — Reporte completo

Objetivo: Hacer múltiples JOINS encadenados.

Pregunta: Crea un reporte completo que muestre: nombre del cliente, ciudad, `orden_id`, fecha, nombre del producto, categoría, cantidad, precio unitario, total.

 **Pista:** Haz **INNER JOIN** entre las 3 tablas: `clientes`, `ordenes`, `productos`

Clave del INNER JOIN

Si un registro no tiene coincidencia en la otra tabla, **desaparece del resultado**. Por eso Pedro (sin órdenes) y la orden 1011 (con producto inexistente) no aparecen en los resultados del **INNER JOIN**.

www.LucianoArgolo.com

LEFT JOIN

El **LEFT JOIN** devuelve **todos** los registros de la tabla izquierda, más las coincidencias de la derecha. Donde no hay coincidencia, aparece **NULL**. Ideal para encontrar ausencias.

Ejercicio 3.1 — Todos los clientes con sus órdenes

Objetivo: Entender LEFT JOIN.

Pregunta: Muestra TODOS los clientes, hayan comprado o no. Si tienen órdenes, muestra los detalles. Si no tienen órdenes, muestra NULL.

 **Pista:** Usa **LEFT JOIN** desde **clientes** hacia **ordenes**

 **Reflexión:** ¿Qué cliente aparece con NULL en las columnas de órdenes?

Ejercicio 3.2 — Clientes que nunca compraron

Objetivo: Usar LEFT JOIN para encontrar ausencias.

Pregunta: Encuentra los clientes que NUNCA hicieron una compra.

 **Pista:** Usa **LEFT JOIN** y filtra con **WHERE orden_id IS NULL**

Ejercicio 3.3 — Todos los productos vendidos o no

Objetivo: LEFT JOIN desde la tabla de productos.

Pregunta: Muestra TODOS los productos, hayan sido vendidos o no.

 **Pista:** Usa **LEFT JOIN** desde **productos** hacia **ordenes**

Ejercicio 3.4 — Productos nunca vendidos

Objetivo: Encontrar productos sin ventas.

Pregunta: ¿Qué productos nunca se vendieron?

 **Pista:** **LEFT JOIN** + **WHERE orden_id IS NULL**

W

Patrón clave del LEFT JOIN

El patrón **LEFT JOIN ... WHERE tabla_derecha.id IS NULL** es una de las técnicas más poderosas en SQL para detectar registros huérfanos, ausencias y problemas de integridad en los datos.

RIGHT JOIN

El **RIGHT JOIN** devuelve **todos** los registros de la tabla derecha, más las coincidencias de la izquierda. Es el espejo del **LEFT JOIN**. En la práctica, casi siempre se puede reescribir como un **LEFT JOIN** invirtiendo el orden de las tablas.

Ejercicio 4.1 — Todas las órdenes con cliente

Objetivo: Entender **RIGHT JOIN**.

Pregunta: Muestra **TODAS** las órdenes, tengan cliente válido o no. Si el cliente existe, muestra sus datos. Si no, muestra **NULL**.

 **Pista:** Usa **RIGHT JOIN** desde **clientes** hacia **ordenes**

 **Reflexión:** ¿Qué orden aparece con **NULL** en los datos del cliente?

Ejercicio 4.2 — Órdenes huérfanas

Objetivo: Detectar problemas de integridad referencial.

Pregunta: Encuentra las órdenes que tienen un **cliente_id** que no existe en la tabla de clientes.

 **Pista:** **RIGHT JOIN** + **WHERE c.cliente_id IS NULL**

Ejercicio 4.3 — Equivalencia **LEFT** vs **RIGHT**

Objetivo: Entender que **RIGHT JOIN** se puede reescribir como **LEFT JOIN**.

Pregunta: Reescribe el ejercicio 4.1 usando **LEFT JOIN** en vez de **RIGHT JOIN** (invirtiendo el orden de las tablas).

 **Pista:** Cambia el orden: **ordenes LEFT JOIN clientes**

www.LucianoArgolo.com

RIGHT JOIN en la práctica

El **RIGHT JOIN** es válido pero poco usado. La mayoría de los equipos de datos prefieren reescribirlo como **LEFT JOIN** invirtiendo el orden de las tablas, ya que resulta más legible y consistente con el flujo natural de lectura del código SQL.

www.LucianoArgolo.com

FULL OUTER JOIN

El **FULL OUTER JOIN** devuelve **todos** los registros de ambas tablas, haya coincidencia o no. Es la herramienta ideal para auditorías de integridad de datos.

Ejercicio 5.1 — Todos los clientes y todas las órdenes

Objetivo: Entender FULL OUTER JOIN.

Pregunta: Muestra TODOS los clientes Y TODAS las órdenes, haya coincidencia o no.

 **Pista:** Usa FULL OUTER JOIN (o FULL JOIN)

 **Reflexión:** ¿Qué registros aparecen con NULL en algún lado?

Ejercicio 5.2 — Detectar problemas de datos

Objetivo: Usar FULL OUTER JOIN para auditoría de datos.

Pregunta: Encuentra: (1) Clientes sin órdenes y (2) Órdenes sin cliente válido.

 **Pista:** Usa FULL OUTER JOIN y filtra con condiciones IS NULL en cada caso.

  **Caso de uso real:** El FULL OUTER JOIN es muy utilizado en procesos de reconciliación de datos, donde necesitas identificar registros que existen en una fuente pero no en otra (y viceversa).

www.LucianoArgolo.com

FULL OUTER JOIN — La auditoría completa

Combina el poder del LEFT JOIN y el RIGHT JOIN en una sola consulta. Ningún registro queda fuera del resultado, lo que lo convierte en la herramienta perfecta para detectar inconsistencias en los datos.

www.LucianoArgolo.com

SELF JOIN

El **SELF JOIN** une una tabla consigo misma usando **alias diferentes**. Es fundamental para trabajar con jerarquías, relaciones recursivas y comparaciones dentro de la misma tabla.

Ejercicio 6.1 — Jerarquía de empleados

Objetivo: Entender SELF JOIN.

Pregunta: Muestra cada empleado con el nombre de su jefe. Incluye: empleado_id, nombre del empleado, cargo, jefe_id, nombre del jefe.

💡 **Pista:** Haz JOIN de empleados consigo misma, uniendo empleado_id con jefe_id

Ejercicio 6.2 — Empleados sin jefe

Objetivo: Encontrar el nivel más alto de la jerarquía.

Pregunta: ¿Qué empleado no tiene jefe? (el CEO)

💡 **Pista:** LEFT JOIN consigo misma + WHERE jefe_id IS NULL

Ejercicio 6.3 — Subordinados directos

Objetivo: Encontrar relaciones inversas.

Pregunta: Muestra quién reporta directamente a cada jefe.

💡 **Pista:** SELF JOIN donde empleado.jefe_id = jefe.empleado_id

www.LucianoArgolo.com

SELF JOIN — La clave está en los alias

Al hacer un SELF JOIN, debes usar dos alias distintos para la misma tabla (por ejemplo, **e AS empleado** y **e AS jefe**). Sin alias, SQL no puede distinguir a cuál instancia de la tabla te estás refiriendo en cada columna.

www.LucianoArgolo.com

JOINS Múltiples

Combinar múltiples JOINS en una sola consulta es una habilidad esencial en ingeniería de datos. Permite construir reportes ricos a partir de tablas relacionadas.

Ejercicio 7.1 — Reporte completo con 3 tablas

Objetivo: Combinar múltiples JOINS.

Pregunta: Crea un reporte que muestre: nombre del cliente, ciudad, `orden_id`, fecha, nombre del producto, categoría, cantidad, precio unitario, total (cantidad × precio). Ordena por cliente y fecha.

💡 **Pista:** Haz `INNER JOIN` entre las 3 tablas: `clientes`, `ordenes`, `productos`

Ejercicio 7.2 — Resumen por cliente

Objetivo: Agregar después de JOINS.

Pregunta: Para cada cliente, muestra: nombre, ciudad, cantidad total de órdenes, monto total gastado.

💡 **Pista:** Haz `LEFT JOIN` (para incluir clientes sin órdenes), luego `GROUP BY` y `SUM()`

Ejercicio 7.3 — Resumen por producto

Objetivo: Analizar ventas por producto.

Pregunta: Para cada producto, muestra: nombre, categoría, cantidad total vendida, ingresos totales. Incluye productos que nunca se vendieron (con 0).

💡 **Pista:** `LEFT JOIN` desde productos, `GROUP BY`, `COALESCE()` para manejar NULLs

www.LucianoArgolo.com

JOINS Múltiples — Buenas prácticas

Al encadenar varios JOINS, usa siempre alias de tabla cortos y claros. Aplica `COALESCE()` para convertir NULLs en ceros cuando necesites mostrar métricas de registros sin coincidencia. Ordena el resultado para facilitar la lectura.

www.LucianoArgolo.com

Resumen: Tipos de JOINS

Referencia rápida de todos los tipos de JOINS practicados en esta semana:

Tipo	Qué devuelve	Cuándo usarlo
INNER JOIN	Solo coincidencias en ambas tablas	Reporte de datos relacionados
LEFT JOIN	Todo de la izquierda + coincidencias de la derecha	Encontrar ausencias (IS NULL)
RIGHT JOIN	Todo de la derecha + coincidencias de la izquierda	Casi nunca – usar LEFT invertido
FULL OUTER JOIN	Todo de ambas tablas	Auditoría de integridad de datos
SELF JOIN	Una tabla consigo misma (usa alias diferentes)	Jerarquías, relaciones recursivas

Reflexión final sobre JOINS

Dominar los JOINS es dominar el corazón de SQL. La elección del tipo correcto de JOIN determina qué datos aparecen y cuáles se excluyen. Siempre pregúntate: ¿quiero conservar todos los registros de una tabla, o solo los que tienen coincidencia?

Checklist de Finalización

Verifica que completaste todos los ejercicios antes de subir tu notebook:



Tablas temporales

Creé todas las tablas temporales:
clientes, productos, ordenes,
empleados



INNER JOIN

Completé todos los ejercicios de INNER JOIN (2.1, 2.2, 2.3)



LEFT JOIN

Completé todos los ejercicios de LEFT JOIN (3.1, 3.2, 3.3, 3.4)



RIGHT JOIN

Completé todos los ejercicios de RIGHT JOIN (4.1, 4.2, 4.3)



FULL OUTER JOIN

Completé todos los ejercicios de FULL OUTER JOIN (5.1, 5.2)



SELF JOIN

Completé todos los ejercicios de SELF JOIN (6.1, 6.2, 6.3)



JOINS Múltiples

Completé todos los ejercicios de JOINS Múltiples (7.1, 7.2, 7.3)



Comprensión

Entiendo la diferencia entre cada tipo de JOIN y cuándo aplicar cada uno



GitHub

Subí mi notebook completo a GitHub con todos los ejercicios resueltos

Instructor: Luciano Argolo

www.LucianoArgolo.com